

3.2 Context Map

The EcoGrid Energy platform is divided into three contexts: Marketplace, Smart Meter Integration and Financial Settlement. Each of these contexts has a role and is responsible for the data that belongs to its area. This way of organizing things helps to keep the connections, between the parts to a minimum. Lets each area grow without changing the whole system.

The Smart Meter Integration context takes in. Works with energy data that comes from smart meters connected to the internet. It checks the data that comes in and turns it into energy events. It does not have any logic for matching energy offers or handling money; instead it sends out events that other parts can use. The Marketplace context handles energy offers what buyers need what sellers can. Matches them up. It uses the verified energy data from Smart Meter Integration to see if there is energy to trade. It stays separate from the work of processing data from meters.

The Financial Settlement context deals with the money part of energy trades that have been completed. When a trade is approved by the Marketplace, a request for settlement is made. Financial Settlement handles the money part. Sends out the result of the settlement. It does not rely on how the other parts work

These contexts use both APIs and asynchronous events. Synchronous communication is used when a quick reply is needed, like when a person asks for marketplace details. Asynchronous communication is used for a lot of data coming in from meters and for sending messages, between parts. This helps keep the parts separate and lets them work on their own.

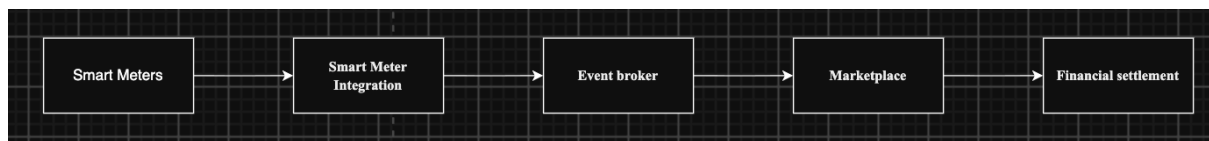


Figure 1: EcoGrid Energy context map and primary cross-domain flow.

Key Integration Relationships

Source Context	Target Context	Main Information	Integration Pattern	Reason
Smart Meter Integration	Marketplace	Validated energy availability	Asynchronous event	Handles high-frequency data without tightly coupling Marketplace
Marketplace	Financial Settlement	Confirmed trade / settlement request	Asynchronous event	Decouples trading from payment processing
Financial Settlement	Marketplace	Settlement result	Asynchronous event	Allows Marketplace to react without direct financial-system dependency
Marketplace	User Interface / API	Offers, matches and trade status	Synchronous API	Immediate response for user interactions
Smart Meter Integration	Monitoring / Operations	Processing status and failures	Events / metrics	Supports monitoring and fault detection

3.3 Organisational Alignment

The architecture is aligned with the structure of the four-person founding engineering team. Instead of creating many independently managed services, responsibilities are organised around the major business domains and shared engineering responsibilities.

Team Responsibility	Main Focus	Related Architecture Area
Member 1	Domain requirements and Marketplace	Marketplace and bounded-context definition
Member 2	Architecture and integration design	Context Map, architecture and integration patterns
Member 3	Resilience and financial processing	Financial Settlement, distributed transactions, fault tolerance and observability
Member 4	Development collaboration and delivery	GitHub, task management, communication and delivery evidence

This ownership model does not mean that everyone works separately. Each person has an area but the group can still look at decisions together and use the shared repository to make the work clear to everyone. The four-person team structure also affects the choice. A distributed microservices architecture could create a lot of extra work.

4. Architectural Nomination and Rationale

4.1 Nominated Style

The nominated architecture, for EcoGrid Energy is a type of system design that focuses on areas of the business. It uses events to communicate between parts of the system. The design makes sure that each part has limits and works on its own. It is not a big system and it is not made up of many small parts either. Instead it divides the business areas and uses event-based communication. This approach helps keep things separate and allows for processing when there is a lot of work to do.

1. Smart Meter Integration
2. Marketplace
3. Financial Settlement

Each domain has its own responsibilities and data ownership. Internal implementation details remain within the relevant context, while communication across boundaries occurs through defined APIs or domain events.

Why this architecture suits EcoGrid

The key architectural challenge comes from the high-frequency nature of IoT meter data. Smart meters can generate energy generation and consumption readings constantly. If every single reading were sent directly through Marketplace it would create coupling. This would make Marketplace reliant, on the ingestion pipeline, which's not ideal.

An event-driven approach allows Smart Meter Integration to receive and process readings independently. Relevant validated information can then be published for consumers. Marketplace therefore receives the information required for trading than every raw reading.

The architecture supports scalability. If the number of smart meters increases Smart Meter Integration can scale without scaling Marketplace or Financial Settlement.

It also improves fault isolation. If Financial Settlement is temporarily unavailable appropriate trading operations can continue while settlement events wait for processing. Likewise a Marketplace issue should not stop Smart Meter Integration from collecting and processing meter data.

The architecture supports evolvability because new consumers can subscribe to events without major changes, to the event producer. For example a future analytics component could consume energy-related events without changing the Smart Meter Integration logic.

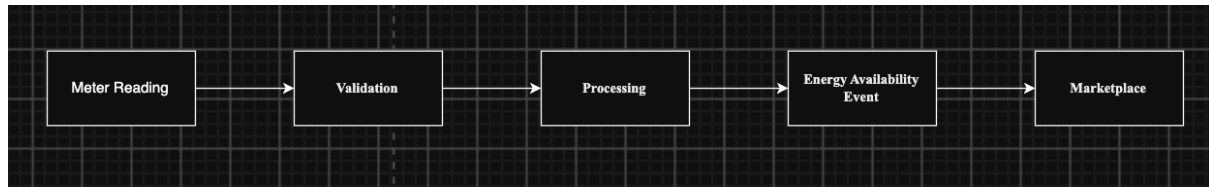


Figure 2: Smart-meter data processing and marketplace event flow.

Synchronous and Asynchronous Communication

Synchronous communication includes things like user requests for marketplace information, authentication and authorization getting the trade status and user-facing API operations.

Asynchronous communication is used when its more important to separate processes and handle a lot of data at once. This includes smart-meter events updates about energy availability, events that happen when a trade is made settlement requests and notifications, about whether a settlement was completed or failed. This mixed method stops us from using one type of communication for everything.

Proposed High-Level Architecture

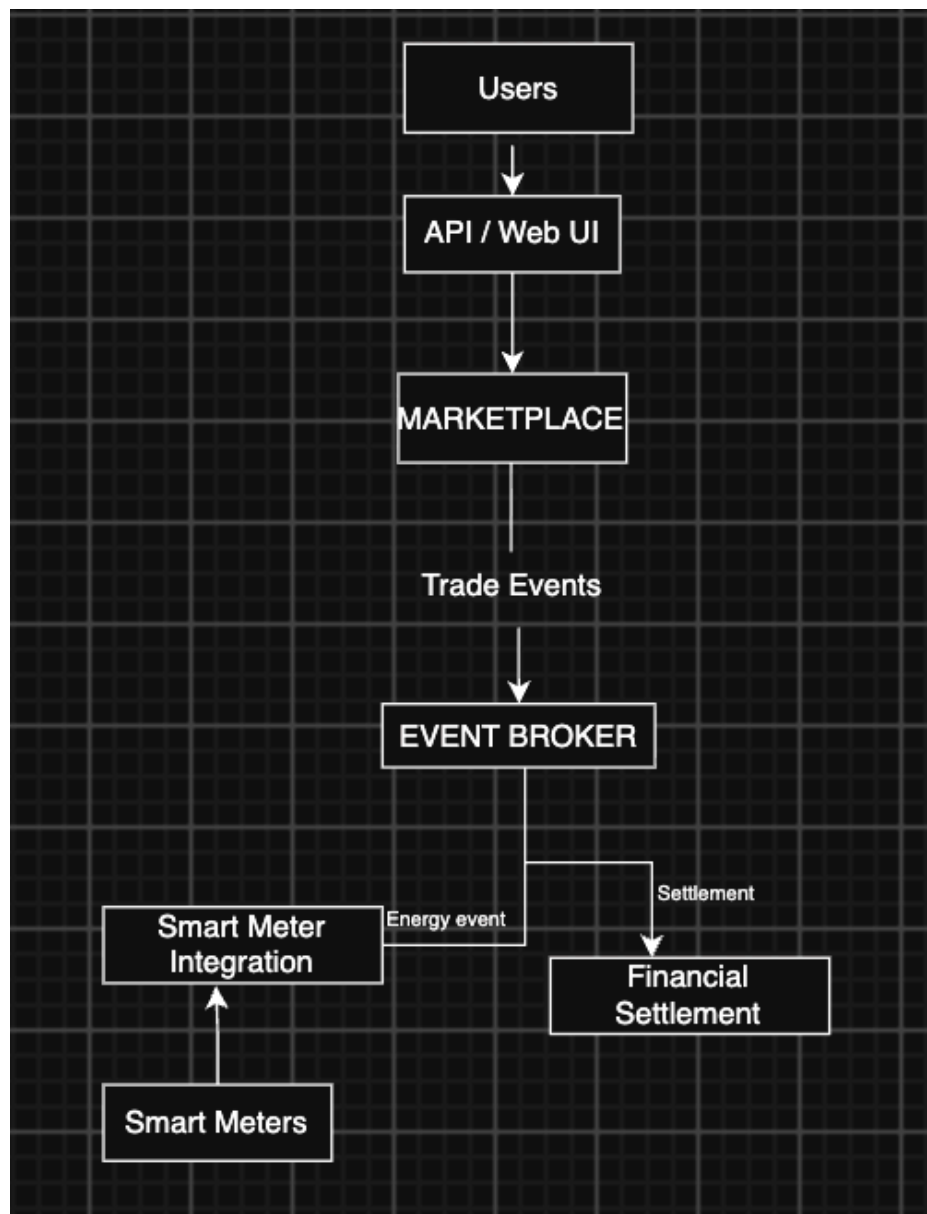


Figure 3: Proposed EcoGrid architecture with event-driven integration (draw.io-style diagram).

The Event Broker is not part of the business domain. It serves as a way to move events, between different parts of a system that work on their own. These parts operate independently. Don't need to wait for each other. When a quick response is needed synchronous APIs are still used. This keeps things simple and scalable. It also prevents splitting services much which could make the system harder to manage.

4.2 Trade-off Analysis

Choosing an architecture is not about getting the highest scalability. A scalable design can also be harder to build, monitor and maintain. For EcoGrid, the choice of architecture needs to balance scalability with the size and capability of the four-person team. The major alternatives considered are Monolith architecture, Modular Monolith architecture, Microservices architecture and Event-Driven Architecture. Event-Driven Architecture is treated as an integration style that can be combined with domain-oriented service boundaries rather than as a direct replacement, for service decomposition.

Criterion	Monolith	Modular Monolith	Microservices	Event-Driven	Proposed EcoGrid
Initial simplicity	High	High	Low	Medium	Medium-High
Domain separation	Low	High	High	High	High
High-frequency IoT handling	Low-Medium	Medium	High	Very High	Very High
Independent scaling	Low	Low-Medium	High	High	High
Operational overhead	Low	Low	High	Medium-High	Medium
Fault isolation	Low	Medium	High	High	High
Team suitability	High	Very High	Medium-Low	Medium	High
Evolvability	Medium	High	High	Very High	Very High
Complexity tax	Low	Low-Medium	High	Medium-High	Controlled

Monolithic Architecture

A traditional monolith would keep Marketplace, Smart Meter Integration and Financial Settlement inside one single deployable application. The biggest benefit is simplicity. Development is easier. Deployment is simpler. Testing locally is more straightforward because there are parts to manage.. There's a downside. The Smart Meter Integration runs at a frequency. This kind of workload can slow down. Affect the other functions in the same application. Scaling becomes a problem too. You can't scale one part. You have to scale the application. That's inefficient. It also means the three main business areas become tightly linked. Changes in one area can have effects across the others. This creates coupling, between them.

Decision: Rejected as the primary architecture.

Modular Monolith

A modular monolith gives internal boundaries while keeping a single deployable application. It gives lower overhead compared to microservices and would be a reasonable early-stage option. Scalability is the main limitation.

Decision: Considered as a possible early-stage implementation, but not selected as the final architecture.

Microservices

This is appealing because the IoT workload might grow in ways.. Microservices also bring a lot of complexity. There are pipelines to manage. Monitoring becomes harder. Distributed tracing adds another layer. Network communication needs handling. Security boundaries must be defined. Versioning gets complicated.. Dealing with distributed failures becomes more difficult. For a founding team of four people having too many services can reduce productivity. It can also increase risk.

Decision: Useful principles adopted, but excessive microservice decomposition is rejected.

Event-Driven Architecture

Event-driven communication is especially suitable for smart-meter information, which is produced at a frequency and does not always need instant synchronous handling. Event-driven communication lets producers and consumers work separately and at their pace.. Event-driven communication adds complexity, such as, in schemas duplicate messages ordering, retries, failures and eventual consistency.

Decision: Accepted as a major integration mechanism, particularly for high-volume IoT and cross-domain events.

Final Architectural Decision

The final decision is to go with a domain-oriented architecture that uses event-driven integration. The design includes:

- Strong boundaries between the three business contexts
- Each domain owns its own data independently
- Event-driven communication is used for high-frequency and cross-domain workflows
- Synchronous APIs are used when immediate responses are needed
- High-load processing components can scale independently
- Service decomposition is kept under control
- Clear ownership is assigned matching the four-person team structure

Key principle: Separate domains where business boundaries are clear, and introduce distributed complexity only where it provides measurable value.

Trading and Settlement Flow



Figure 4: EcoGrid Energy trading and settlement flow.

AI-Assisted Design Evidence Note

The AI-assisted design evidence for this contribution should be compiled in Section 7 of the final group report using the actual conversation history for: (1) Context Map, (2) Architectural Nomination, and (3) Trade-off Analysis. The evidence should include the initial prompt, three progressive refinements, critical evaluation, final validated outcome, and genuine screenshots. No AI evidence has been fabricated in this document.

Today 12:34 PM

Design a context map for EcoGrid Energy using three bounded contexts: Marketplace, Smart Meter Integration, and Financial Settlement. Explain how they should communicate.

A simple context map for **EcoGrid Energy** can use three bounded contexts:

- 1. Smart Meter Integration**
Responsible for receiving and validating high-frequency electricity data from smart meters.
- 2. Marketplace**
Responsible for energy offers, buyer/seller matching, and confirming energy trades.
- 3. Financial Settlement**
Responsible for processing the financial settlement after a trade is confirmed.

Communication
A suitable flow is:

Smart Meters → Smart Meter Integration → Event Broker → Marketplace → Event Broker → Financial Settlement

- **Smart Meter Integration → Marketplace:** asynchronous events, such as `EnergyAvailability`.
- **Marketplace → Financial Settlement:** asynchronous events, such as `TradeConfirmed` or `SettlementRequested`.
- **Financial Settlement → Marketplace:** settlement success/failure events.
- **User → Marketplace:** synchronous API communication when an immediate response is needed.

This keeps IoT processing separate from Marketplace logic, while the Event Broker allows the domains to exchange information without becoming tightly coupled.

Design a context map for EcoGrid Energy using three bounded contexts: Marketplace, Smart Meter Integration, and Financial Settlement. Explain how they should communicate.

EcoGrid Context Map

- 1. Smart Meter Integration**
Collects and validates high-frequency IoT energy data.
- 2. Marketplace**
Matches energy sellers with buyers and confirms trades.
- 3. Financial Settlement**
Processes payments and settlement after confirmed trades.

Communication Flow:

Smart Meters → Smart Meter Integration → Event Broker → Marketplace → Event Broker → Financial Settlement

- **Smart Meter → Marketplace:** asynchronous events such as `EnergyAvailability`.
- **Marketplace → Financial Settlement:** asynchronous `TradeConfirmed` events.
- **Financial Settlement → Marketplace:** settlement success/failure events.
- **Users → Marketplace:** synchronous API for immediate requests.

This separation reduces coupling and allows each domain to scale independently.

📄 🗨️ ⬆️ ⬇️ ↺ ...

Refine this design considering that Smart Meter Integration receives high-frequency IoT data. Which communication should be synchronous and which should be asynchronous? more shortly

Because smart meters generate **high-frequency data**, most communication should be asynchronous.

- **Smart Meter Integration → Marketplace:** Asynchronous events
- **Marketplace → Financial Settlement:** Asynchronous events
- **Financial Settlement → Marketplace:** Asynchronous events
- **User → Marketplace:** Synchronous API

This reduces system load, improves scalability, and keeps the domains loosely coupled.

Refine it for a four-person engineering team. Avoid unnecessary service complexity.

For a **four-person team**, keep only the three main domains:

Smart Meter Integration → Event Broker → Marketplace → Financial Settlement

Use:

- **Asynchronous events** for IoT and trade processing.
- **Synchronous APIs** only when immediate responses are needed.
- Avoid creating many small microservices.

This keeps EcoGrid **simple, scalable, and manageable for a small team**.

📄 🗑️ ⬆️ 🔊 🔁 ⋮

Give me the final validated context map showing the three bounded contexts, Event Broker, communication direction, and purpose of each connection.

Final Validated Context Map

Smart Meters → Smart Meter Integration → Event Broker → Marketplace → Financial Settlement

- **Smart Meter Integration → Event Broker:** publishes validated energy data.
- **Event Broker → Marketplace:** sends `EnergyAvailability` events.
- **Marketplace → Financial Settlement:** sends confirmed trade/settlement events.
- **Financial Settlement → Marketplace:** sends settlement success/failure events.
- **User ↔ Marketplace:** synchronous API for immediate requests.

This design keeps the three domains **separate, scalable, and manageable for a four-person team**.

What architecture would you recommend for EcoGrid Energy, which has a Marketplace, high-frequency Smart Meter Integration, and Financial Settlement?

I recommend a **domain-oriented architecture with event-driven integration**.

- **Smart Meter Integration:** processes high-frequency IoT data.
- **Marketplace:** manages energy trading and matching.
- **Financial Settlement:** handles payments.
- **Event Broker:** connects the domains asynchronously.
- **Synchronous APIs:** used when immediate responses are needed.

This provides good **scalability, separation, and maintainability** without unnecessary complexity.

📄 🗑️ ⬆️ 🔊 🔁 ⋮

Compare event-driven architecture, microservices, modular monolith and monolith for EcoGrid's high-frequency IoT requirement.

Architecture	EcoGrid suitability
Monolith	Simple, but weak for independently scaling high-frequency IoT processing.
Modular Monolith	Good separation and low complexity, but limited independent scaling.
Microservices	Excellent scalability, but high operational complexity for a small team.
Event-Driven	Very suitable for high-frequency IoT data because it supports asynchronous processing and loose coupling.

Best choice: Event-driven integration with clear domain boundaries, without creating unnecessary microservices.

Consider EcoGrid has only a four-person engineering team. Compare the complexity and operational overhead of these options.

For a **four-person team**:

- **Monolith**: Lowest complexity, but limited scalability.
- **Modular Monolith**: Low complexity with good domain separation.
- **Microservices**: High operational overhead; too complex for a small team.
- **Event-Driven**: Medium complexity but excellent for high-frequency IoT data.

Best balance: Domain-oriented design with **event-driven integration**, while avoiding too many microservices.

📄 🗑️ ⬆️ 🔊 🔁 ...

Give a final architecture recommendation balancing scalability, maintainability, fault isolation and simplicity for EcoGrid.

Final Recommendation

Use a **domain-oriented architecture with event-driven integration**.

- **Scalability**: Smart Meter Integration can scale independently.
- **Maintainability**: Clear separation between the three domains.
- **Fault isolation**: Failure in one domain does not stop the whole system.
- **Simplicity**: Avoids unnecessary microservices for the four-person team.

This gives EcoGrid a good balance between **performance, flexibility, and manageable complexity**.

Add complexity tax to the comparison and explain why a four-person engineering team should avoid unnecessary architectural complexity.

Adding **complexity tax** changes the comparison:

- **Monolith**: Low complexity tax, but poor independent scalability.
- **Modular Monolith**: Low complexity tax with good maintainability.
- **Microservices**: High complexity tax due to deployment, networking, monitoring, and service coordination.
- **Event-Driven**: Medium complexity tax because of event management, retries, and message handling.

For a **four-person team**, unnecessary microservices should be avoided because the operational workload could become difficult to manage. EcoGrid should add distributed complexity only where it provides clear benefits.

📄 🗑️ ⬆️ 🔊 🔁 ...

Provide a final trade-off analysis and recommend the most appropriate architecture for EcoGrid, considering high-frequency IoT data and evolutionary architecture.

Final Trade-off Analysis

For EcoGrid:

- **Monolith**: Simple but weak for independent scaling.
- **Modular Monolith**: Maintainable and simple, but less flexible for high-frequency IoT workloads.
- **Microservices**: Highly scalable but adds too much complexity for a four-person team.
- **Event-Driven**: Handles high-frequency IoT data efficiently and supports loose coupling and fault isolation.

Final recommendation: Use a **domain-oriented architecture with event-driven integration**. It supports EcoGrid's future evolution while keeping the current system manageable for a small engineering team.

📄 🗑️ ⬆️ 🔊 🔁 ...

Compare monolith, modular monolith, microservices and event-driven architecture for the EcoGrid Energy platform.

Architecture	Trade-off for EcoGrid
Monolith	Simple to build, but difficult to scale IoT processing independently.
Modular Monolith	Better separation and easier maintenance, but limited independent scaling.
Microservices	Strong scalability and fault isolation, but adds high operational complexity.
Event-Driven	Well suited to high-frequency IoT data, with asynchronous processing and loose coupling.

For EcoGrid, **event-driven integration with clear domain boundaries** provides the best balance without unnecessary complexity.

Compare these options using scalability, maintainability, operational overhead, fault isolation and suitability for EcoGrid.

Criteria	Monolith	Modular Monolith	Microservices	Event-Driven
Scalability	Low	Medium	High	High
Maintainability	Medium	High	High	High
Operational Overhead	Low	Low	High	Medium
Fault Isolation	Low	Medium	High	High
EcoGrid Suitability	Low	Good	Good but complex	Very Good

For EcoGrid, **event-driven integration** is most suitable because it supports high-frequency IoT data while providing scalability and fault isolation with manageable operational overhead.